

An Improved Method for Building Sets of Permutation-Fair Dice

Michael Purcell

Abstract. We define what it means for a set of dice to be m/n permutation-fair. We show how some collections of sets of m/n permutation-fair dice can be combined to create new sets of dice that are $(m+1)/n$ permutation fair. By applying this method recursively, we show that our method is generic. That is, it can be used to create sets of dice that are m/n permutation fair for all m and n . We show that our method is more efficient than what we call the concatenation construction, which is the only other known generic method for constructing sets of permutation-fair dice. We conclude by demonstrating how our method can be used to construct a set of five 120-sided dice that is permutation fair.

1. INTRODUCTION Many tabletop games are designed so that players take *turns* interacting with the game’s mechanisms. As such, the players need some way to determine the order in which they will take their turns, i.e. the *turn order*. For some games, the turn order can significantly affect how the players will play the game and the players will need a way to determine the turn order that is “fair”.

One option is to have each player roll a standard die. If no two players roll the same number then the results of their rolls can be used to determine a turn order, perhaps by specifying that players should take turns in descending order according to the values of their rolls. If two or more players do roll the same number, then those players can break the tie by rolling their dice again. With high probability, only a few rounds would be required to determine a complete turn order.

Suppose that, instead of using standard dice, each player uses a die from a set of nonstandard dice with the following properties:

- No number appears on more than one face of any die in the set, so rolling this set of dice always yields a result with n distinct numbers.
- The face values are arranged such that if all n dice are rolled and then sorted according to the resulting values, then all $n!$ permutations of the dice are equally likely.

We will say that such a set of dice is *permutation fair*. By using a set of dice that is permutation fair, the players can determine the turn order for their game in a single round. Table 1 gives one example of a set $\mathcal{G}$ of dice that is permutation fair. This set was discovered by Eric Harshbarger and Robert Ford in 2010 (see [11]).

Table 1. A set of four 12-sided dice that are permutation fair.

Die	Faces											
	i	ii	iii	iv	v	vi	vii	viii	ix	x	xi	xii
a	1	8	11	14	19	22	27	30	35	38	41	48
b	2	7	10	15	18	23	26	31	34	39	42	47
c	3	6	12	13	17	24	25	32	36	37	43	46
d	4	5	9	16	20	21	28	29	33	40	44	45

Notice that if there are fewer than four players, then they can still use $\mathcal{G}$ to determine a turn order. This is because if a set of dice is permutation fair, then every subset of that set of dice is permutation fair as well. But what if there are more than four players? It has been shown (see [14]) that there are sets of n dice that are permutation fair for all $n \in \mathbb{N}$. So, in principle, by using an appropriate set of permutation-fair dice any number of players can determine a turn order. In practice, the situation is more complicated because the best known generic constructions for permutation-fair sets produce dice with a prohibitively large number of faces.

For example, the construction in [14] produces a set of n dice that have $(n!)^{n-1}$ faces. When $n = 4$ this construction therefore yields a set of four 13,824-sided dice. When $n = 5$ it yields a set of five 207,360,000-sided dice. Recall, however, that we have seen that there exists a set of four 12-sided dice that are permutation fair. It is then natural to ask, for each n , what is the minimum number of sides necessary to construct a set of n permutation-fair dice? Can we find examples that realize this lower bound? If not, what are the smallest examples that we can find?

Our Contributions In this paper, we will explore the answers to these questions. We will introduce a novel technique that can be used to construct sets of relatively small dice that are permutation fair. In particular, we will describe how this technique can be used to construct a set of five 120-sided dice that is permutation fair. To the best of our knowledge, our construction is the most efficient known construction for a set of five dice that are permutation fair. The best previously known construction (see [11]) requires five 180-sided dice while the generic construction mentioned above would require five 207,360,000-sided dice.

Our approach to the problem involves working with sets of dice that satisfy weaker symmetry constraints than those required for permutation fair dice. More precisely, we consider sets of n dice that have the property that every subset of m of those dice is permutation fair. We say that such a set of dice is m/n permutation fair. Observe that the property that we have called permutation fairness to this point is equivalent to a set of dice being n/n permutation fair. Our construction is predicated on the observation that, by combining carefully chosen sets of dice that are m/n permutation fair, we can construct a set of dice that is $(m+1)/n$ permutation fair.

We formulate the problem of choosing sets of dice that are m/n permutation fair to work with as the solution to a mixed integer linear programming (ILP) problem. This allows us to use existing implementations of algorithms to solve ILP problems to find suitable sets of dice or prove that no such sets exist. In our work, we used the ILP solver CBC (see [2]). We used the Python package PuLP (see [5]) to describe our problems, interface with the ILP solver, and process the resulting output.

Practical Considerations There are many other efficient ways in which a group of players could fairly determine a turn order. These include simply identifying each player with a unique card, shuffling those cards together, and then dealing the cards out one at a time to determine the turn order. This protocol is simple, efficient, and will never require more than a single round to execute. Even the protocol using standard dice described above will be sufficient in most cases because the probability that more than a few rounds will be required is vanishingly small.

While the motivating problem might be more easily solved by the other methods suggested, there is still an important distinction which should be made. Fairly determining a turn order is a proxy for a particular kind of resource allocation problem. At the conceptual level, permutation-fair dice are a solution for an environment which has no central authority and which requires an asynchronous protocol with fixed runtime.

2. RELATED WORK According to [11], the earliest work on permutation-fair dice was motivated by a question asked by James Ernest in 2010. He asked if it might be possible to construct a set of dice with somewhat nebulously defined “go first” properties. Eric Harshbarger and Robert Ford started working on the problem at that time and together are responsible for many of the fundamental results in the field.

Types of Fairness Among their earliest contributions was to establish a taxonomy of three different kinds of fairness that a set of “go first dice” might exhibit.

1. *Go-first fairness*: A set of dice is go-first fair if, when all of the dice in the set are rolled, every die has an equal probability of displaying the largest value.
2. *Place fairness*: A set of dice is place fair if, when all of the dice in the set are rolled and then sorted according to the resulting values, every die has an equal probability of ending up in each place in the resulting ordering.
3. *Permutation fairness*: A set of dice is permutation fair if, when all of the dice are rolled and then sorted according to the resulting values, every possible ordering of the dice occurs with equal probability.

Notice that permutation fairness implies place fairness and place fairness implies go-first fairness. As such, as discussed in the introduction, we will focus our attention on the search for sets of dice that are permutation fair.

Size Constraints During their initial investigations, Harshbarger and Ford also identified the following condition that a set of permutation-fair dice must satisfy.

Proposition 1. *If $\mathcal{S}$ is a set of n d -sided dice that is permutation fair and $\{p_i\}$ are the distinct prime factors of $n!$, then $\prod_i p_i \mid d$.*

Proof. There are d^n equally likely possible outcomes when the dice in $\mathcal{S}$ are rolled. Because $\mathcal{S}$ is permutation fair, we can partition these outcomes into $n!$ equal parts and so we must have $n! \mid d^n$. So, for all i we must also have $p_i \mid d^n$. Therefore, for all i we have $p_i \mid d$ and the result follows immediately. ■

Among other things, Proposition 1 sets a lower bound for the size of the dice in a set that is permutation fair. Notably, we see that if $\mathcal{S}$ is a set of five d -sided dice that is permutation fair, then $d \geq 30$.

Construction Methods The first sets of permutation-fair dice were discovered using a combination of computer searches and bespoke mathematical analyses (see [11]). Since that time, however, at least three construction methods have been devised.

- *Induction Construction*: This construction (see [6, 10]) provides a way to augment a set of n/n permutation fair dice to create a set of $(n+1)/(n+1)$ permutation-fair dice. A curious feature of the sets of dice created via this method is that they are often *inhomogeneous*. That is, sets that can be realized using dice that are not all the same size. The induction construction is believed to have been originally used by Paul Vanetti before being described more fully by Robert Ford.
- *Binary Construction*: This construction (see [6]) identifies a class of dice that can be represented by an array of bits. It can be shown that there are constraints that such an array must satisfy if it is to represent a set of dice that are go-first fair. Furthermore, it is possible to efficiently search the space of candidates that satisfy these constraints to identify sets of binary dice that are place fair or permutation fair. The binary construction was originally described by James Grime and Brian Pollock who used this method to find a set of five 60-sided dice that are place fair.

- **Concatenation Construction:** This construction (see [14]) provides a way to augment a set of m/n permutation-fair dice to create a set of $(m+1)/n$ permutation-fair dice. This method can be used to build sets of n d -sided dice that are m/n permutation fair where $d = (n!)^{m-1}$. The concatenation construction was originally described by Tomáš Gavenčiak and Václav Rozhoň.

The techniques described in this paper are a refinement of the concatenation construction. The concatenation construction can be understood as a sequence of operations, each of which multiplies the number of sides on the dice in the resulting set by $n!$. We show how to reduce the cost of some of these operations. As an extreme example, consider the case of augmenting a set of $3/n$ permutation-fair dice to create a set of $4/n$ permutation-fair dice. Using the concatenation construction, the multiplier associated with this operation would be $n!$. Using our improvements, however, for all n the multiplier associated with this operation is 2.

Other Work Permutation-fair dice are a subclass of nonstandard dice. While the study of permutation-fair dice is relatively new field, there is an extensive body of work addressing the properties of other types of nonstandard dice.

One particularly rich vein can be traced back to a series of three papers by Steinhaus and Trybuła in the 1960s (see [18], [19], and [20]). These papers establish some fundamental properties of what are known as *nontransitive dice*. This work was popularized by Martin Gardner in an article written for the Mathematical Games column in Scientific American magazine in 1970 (see [7]).

Nontransitive dice are often used as an example of a “paradoxical” phenomenon that can occur when considering three or more random variables. To wit, suppose that for all pairs of random variables (X, Y) we write $X > Y$ if $\mathbf{P}\{X > Y\} > 1/2$. If X and Y represent two dice, then the statement $\mathbf{P}\{X > Y\} > 1/2$ can be interpreted as meaning that when the two dice are rolled and the resulting values compared, die X will “beat” die Y more than half of the time. If $\mathcal{S} = \{A, B, C\}$ is a set of dice such that $A > B$, $B > C$, and $C > A$, then we say that $\mathcal{S}$ is nontransitive.

Of the papers that discuss nontransitive dice, those that are most closely related to the study of permutation-fair dice are those that are concerned with the probability that two dice will be *tied*. That is, sets of dice $\mathcal{S}$ such such that $\mathbf{P}\{X > Y\} = 1/2$ for all $X, Y \in \mathcal{S}$. These include the work of Traldi et al (see [3], [13]), Schaefer et al (see [16], [17]), and Hur and Kim (see [12]). Notice that $\mathcal{S}$ is a set of n tied dice if and only if it is $2/n$ permutation fair.

One limitation of the work on tied dice is that it is concerned only with the properties of pairs of dice. With permutation-fair dice, we are concerned with the properties of an entire set of dice. If we return our attention to the more general study of nonstandard dice, however, we can find some studies of nontransitive dice that work with the joint properties of large ensembles of dice. Most prominent among these are a series of papers that examine the relationships between various sums of multiple copies of each die in a set. These include the work of James Grime (see [8]) and that of Buhler, Graham, and Hales (see [1]).

Physical Dice An appealing feature of the permutation-fair dice described in Table 1 is that they can be realized using a physical set of four 12-sided dice. It has been shown (see [4]) that isohedra can be used to create physical dice that are fair in the sense that they have the same chance on landing on each of face after a vigorous roll. Recall that the disdyakis tricontahedron is a Catalan solid that has one hundred twenty faces. Because every Catalan solid is an isohedron ([9]), we see that our set of five 120-sided permutation-fair dice can be realized as a physical set of fair dice as well.

3. NOTATION AND TERMINOLOGY In what follows, it will be useful to adopt a different representation of a set of dice. Recall that we have assumed that no number appears on more than one face of any die in a given set. Also, we are only concerned with the relative values on the faces of the dice. So, we can assume that all of the face values in a set of n d -sided dice are drawn from the set $\{1, 2, \dots, nd\}$.

String Representations Given a set $\mathcal{S}$ of dice, we can define a *string representation* $\mathbf{s} = (s_1, \dots, s_r)$ of $\mathcal{S}$. To do so, we first assign a distinct label for each die in $\mathcal{S}$. Then, we let the i th character of $\mathbf{s}$ be the label for the die on which the value i appears.

- If $\mathbf{s}$ is a string of length r , then we will write $\mathbf{s}_{\leq i} = (s_1, \dots, s_i)$ to denote the first i characters of $\mathbf{s}$, $\mathbf{s}_{> i} = (s_{i+1}, \dots, s_r)$ to denote the last $r - i$ characters of $\mathbf{s}$, and $\mathbf{s}_{i_1 \leq j \leq i_2} = (s_{i_1}, \dots, s_{i_2})$ to denote a contiguous substring of $\mathbf{s}$.
- If $\mathbf{s}$ is a string, and σ is a permutation on the characters that comprise $\mathbf{s}$, then we will write $\sigma(\mathbf{s}) = (\sigma(s_1), \dots, \sigma(s_r))$ to denote a *relabelling* of $\mathbf{s}$.
- If $\mathbf{s}$ is a string, then we will write $\mathbf{s}' = (s_r, \dots, s_1)$ to denote the *reverse* of $\mathbf{s}$.
- If $\mathbf{s}$ and $\mathbf{t}$ are any two strings, then we will write $\mathbf{s}||\mathbf{t} = (s_1, \dots, s_r, t_1, \dots, t_r)$ to denote the concatenation of $\mathbf{s}$ and $\mathbf{t}$. If $\mathcal{S}$ and $\mathcal{T}$ are sets of dice with string representations $\mathbf{s}$ and $\mathbf{t}$ respectively, then we will write $\mathcal{S}||\mathcal{T}$ to denote the set of dice with string representation $\mathbf{s}||\mathbf{t}$.

Example. Consider the set $\mathcal{G}$ of dice described in Table 1. If $\mathbf{g}$ is the string representation of $\mathcal{G}$ then we have

$$\mathbf{g} = \text{abcdcdcb} \text{a dbaccabd cbaddabc cbaddabc dbaccabd abcdcdcb} \text{a}.$$

Patterns It will also be useful for us to be able to describe the outcome when we roll some subset of a given set of dice. We will say that $\mathbf{x}$ is a *pattern* of length m if $\mathbf{x}$ is a string comprised of a single instance of m distinct labels.

- If $\mathbf{x}$ is a pattern, then we will write $|\mathbf{x}|$ to denote the length of $\mathbf{x}$.
- If $\mathbf{x}$ is a pattern and $\mathbf{s}$ is a string, then we will say that $\mathbf{x}$ *appears* in $\mathbf{s}$ if we have $\mathbf{x} = (s_{i_1}, \dots, s_{i_m})$ with $i_1 < \dots < i_m$.
- If $\mathbf{x}$ is a pattern and $\mathbf{s}$ is a string, then we will write $(\mathbf{x}; \mathbf{s})$ to denote the number of times $\mathbf{x}$ appears in $\mathbf{s}$. That is, $(\mathbf{x}; \mathbf{s}) = |\{i_1 < \dots < i_m \mid \mathbf{x} = (s_{i_1}, \dots, s_{i_m})\}|$.

To compute the number of times a pattern appears in a given string, we can count how many appearances use each instance of the pattern's last character. The total number of appearances is the sum of these counts. That is, if $|\mathbf{x}| = m$ then we have

$$(\mathbf{x}; \mathbf{s}) = \sum_{i: s_i = x_m} (\mathbf{x}_{\leq m-1}; \mathbf{s}_{\leq i-1}). \quad (1)$$

So, by recursively applying (1), we can efficiently compute $(\mathbf{x}; \mathbf{s})$. Given a set $\mathcal{S}$ of dice with string representation $\mathbf{s}$, we can verify that $\mathcal{S}$ is m/n permutation fair by computing $(\mathbf{x}; \mathbf{s})$ for all $\mathbf{x}$ with $|\mathbf{x}| = m$.

Permutation Fairness Using the language of strings and patterns, we can make a precise statement of what it means for a set of dice to be permutation fair.

Definition. We will say that a set $\mathcal{S}$ of n d -sided dice with string representation $\mathbf{s}$ is m/n permutation fair if, for every $\mathbf{x}$ such that $|\mathbf{x}| = m$, we have

$$(\mathbf{x}; \mathbf{s}) = \frac{d^m}{m!}. \quad (2)$$

4. AN IMPROVED CONCATENATION CONSTRUCTION Before we state our main theoretical result, it will be useful to consider a simple motivating example which was discovered by Eric Harshbarger and Robert Ford in 2010 (see [11]). Note that this while this example is useful for illustrating the concepts that we in use in our construction, it was found using an ad hoc manual search rather than by the methods we describe in this paper.

Example. Consider the set $\mathcal{F}$ of $3/3$ permutation-fair dice described in Table 2 that has string representation

$$\mathbf{f} = \text{abccba bcaacb cabbac}.$$

Notice that $\mathbf{f}$ can be decomposed into three blocks of six characters and that each block is a palindrome in which each label appears twice. In fact, each block is the string representation of a set of dice that is $2/3$ permutation fair. In [14], the authors describe a generic construction for creating $(m+1)/n$ permutation-fair sets of dice by concatenating $n!$ different sets of m/n permutation fair dice. In this case, $n! = 6$ but only three sets of $2/3$ permutation-fair dice were required to create a set of $3/3$ permutation-fair dice. To understand why, we counted how many times each pattern of length three appeared in each block. The resulting values are described in Table 3. Notice that the sum of the values in each row of Table 3 is the same.

Table 2. A set $\mathcal{F}$ of three 6-sided dice that are permutation fair.

Die	Faces					
	i	ii	iii	iv	v	vi
a	1	6	9	10	14	17
b	2	5	7	12	15	16
c	3	4	8	11	13	18

Table 3. Counts for each three-long pattern in the blocks of $\mathbf{f}$.

Pattern	Counts		
	$\mathbf{f}_{1 \leq 6}$	$\mathbf{f}_{7 \leq 12}$	$\mathbf{f}_{13 \leq 18}$
abc	2	2	0
acb	2	0	2
bac	0	2	2
bca	2	0	2
cab	0	2	2
cba	2	2	0

While the three blocks that comprise $\mathbf{f}$ in the preceding example are not $3/3$ permutation fair, they have been carefully chosen to be compatible. That is, the ways in which they are unfair cancel out when they are combined via concatenation. The following theorem extends this idea to the general case. It provides a criterion which ensures that a collection of m/n permutation-fair sets of dice can combined via concatenation to create a set of $(m+1)/n$ permutation-fair dice.

Theorem 2 (Lifting Theorem). Suppose that for all $1 \leq i \leq k$, $\mathcal{S}_i$ is an m/n permutation-fair set of d_i -sided dice with string representation $\mathbf{s}_i$. If there exists a constant C such that for all patterns $\mathbf{x}$ with $|\mathbf{x}| = m + 1$ we have

$$\sum_{i=1}^k (\mathbf{x}; \mathbf{s}_i) = C, \quad (3)$$

then $\mathcal{S} = \mathcal{S}_1 \parallel \dots \parallel \mathcal{S}_k$ is a set of dice that is $(m + 1)/n$ permutation fair.

Proof. Let $\mathbf{s} = \mathbf{s}_1 \parallel \mathbf{s}_2 \parallel \dots \parallel \mathbf{s}_k$. We can decompose the quantity $(\mathbf{x}; \mathbf{s})$ into a sum of products by grouping the appearances of $\mathbf{x}$ according to which part of $\mathbf{s}$ that each part of $\mathbf{x}$ appears in. That is, suppose that the first i_1 components of $\mathbf{x}$ appear in $\mathbf{s}_1$, the next i_2 components of $\mathbf{x}$ appear in $\mathbf{s}_2$, and so on. Using this decomposition we see that

$$(\mathbf{x}; \mathbf{s}) = \sum_{\substack{i_1 + \dots + i_k = m+1 \\ 0 \leq i_1, i_2, \dots, i_k \leq m+1}} (\mathbf{x}_{1 \leq j \leq i_1}; \mathbf{s}_1) (\mathbf{x}_{i_1 < j \leq i_2}; \mathbf{s}_2) \dots (\mathbf{x}_{i_{k-1} < j \leq i_k}; \mathbf{s}_k). \quad (4)$$

We can use the fact that each part of $\mathbf{s}$ is m/n permutation fair to compute the contribution of each partition with $0 \leq i_1, i_2, \dots, i_k \leq m$. For such partitions,

$$(\mathbf{x}_{1 \leq j \leq i_1}; \mathbf{s}_1) (\mathbf{x}_{i_1 < j \leq i_2}; \mathbf{s}_2) \dots (\mathbf{x}_{i_{k-1} < j \leq i_k}; \mathbf{s}_k) = \frac{d_1^{i_1}}{i_1!} \frac{d_2^{i_2}}{i_2!} \dots \frac{d_k^{i_k}}{i_k!}. \quad (5)$$

In particular, (5) shows that the contributions of partitions that are distributed across more than one part of $\mathbf{s}$ do not depend on $\mathbf{x}$. So, we can conclude that for all patterns $\mathbf{x}$ with $|\mathbf{x}| = m + 1$ there exists some constant D such that

$$(\mathbf{x}; \mathbf{s}) = D + \sum_{i=1}^k (\mathbf{x}; \mathbf{s}_i) = D + C. \quad (6)$$

Therefore, each pattern of length $m + 1$ appears the same number of times and we can conclude that $\mathcal{S}$ is $(m + 1)/n$ permutation fair. ■

5. FINDING COMPATIBLE SETS OF DICE We now turn our attention to the task of actually finding new sets of permutation fair dice. Notice that while Theorem 2 provides a guarantee that certain sets of m/n permutation-fair dice can be combined to create a set of $(m + 1)/n$ permutation-fair dice, it does not provide a way to find such compatible sets of dice. Now, we will describe several techniques for doing so.

Analytic Techniques Our first approach is predicated on the fact that the labels that we assign to the dice in a given set are arbitrary. This implies that if $\mathbf{s}$ is the string representation for a set $\mathcal{S}$ of dice, then $\sigma(\mathbf{s})$ is the string representation for a set $\mathcal{T}$ of dice that is equivalent in some sense to $\mathcal{S}$. In particular, $\mathcal{T}$ will have the same fairness properties as $\mathcal{S}$. More precisely, we have the following proposition.

Proposition 3 (Relabelling Theorem). If $\mathcal{S}$ is a set of dice that has string representation $\mathbf{s}$ and σ is a permutation on the set of characters that comprise $\mathbf{s}$, then $\mathbf{s}$ is m/n permutation fair if and only if $\sigma(\mathbf{s})$ is m/n permutation fair.

Proof. Because for all patterns $\mathbf{x}$ we have $(\mathbf{x}; \sigma(\mathbf{s})) = (\sigma^{-1}(\mathbf{x}), \mathbf{s})$, the result follows immediately from the definition of m/n permutation fairness. ■

Proposition 3 provides a way to generate multiple sets of m/n permutation fair dice from a single *seed*. As described in [14], the set of all possible permutations applied to a set of m/n permutation fair dice can be combined via concatenation to form a new, and much larger, set of dice that is $(m + 1)/n$ permutation fair. The following theorem is a restatement of this result using the language developed in this paper.

Theorem 4. *Suppose that $\mathcal{S}$ is a set of m/n permutation-fair dice with string representation $\mathbf{s}$, and that $\{\sigma_i\}_{i=1}^{n!}$ is the set of all permutations on the characters that comprise $\mathbf{s}$. If $\mathbf{t} = \sigma_1(\mathbf{s}) \parallel \cdots \parallel \sigma_{n!}(\mathbf{s})$, then $\mathbf{t}$ is the string representation of a set of dice that is $(m + 1)/n$ permutation fair.*

Proof. It suffices to show that $(\mathbf{x}; \mathbf{t})$ is the same for all $\mathbf{x}$ with $|\mathbf{x}| = m + 1$. Notice that for all i and $\mathbf{x}$, $(\mathbf{x}; \sigma_i(\mathbf{s})) = (\sigma_i^{-1}(\mathbf{x}); \mathbf{s})$. Because $i \mapsto \sigma_i(\mathbf{x})$ is a $(n - |\mathbf{x}|)!$ -to-1 map, there exists a C such that for all $\mathbf{x}$ with $|\mathbf{x}| = m + 1$ we have

$$\sum_{i=1}^{n!} (\mathbf{x}; \sigma_i(\mathbf{s})) = \sum_{i=1}^{n!} (\sigma_i^{-1}(\mathbf{x}); \mathbf{s}) = C. \quad (7)$$

The result then follows from Proposition 3 and Theorem 2. ■

As we have stated earlier, Theorem 4 provides a way to construct set of dice that are m/n permutation fair for arbitrary m and n . Because the dice in the resulting sets will be quite large, however, we would like to find efficient alternative construction methods. Rather than trying to find alternatives that are generic, we will focus our attention on two special cases.

Consider the problem of how to build a set of $2/n$ permutation-fair dice out of k sets of $1/n$ permutation-fair dice. The following theorem shows that this can be done with $k = 2$ by concatenating a string and its reverse to form a palindrome. This theorem provides a possible explanation for the ubiquity of palindromic blocks that we have observed in both the set $\mathcal{G}$ of $4/4$ permutation-fair dice described in Table 1 and the set $\mathcal{F}$ of $3/3$ permutation-fair dice described in Section 4.

Theorem 5. *Suppose that $\mathcal{S}$ is a set of d_s -sided dice that has string representation $\mathbf{s} = (s_1, s_2, \dots, s_r)$. Let $\mathbf{t} = \mathbf{s} \parallel \mathbf{s}'$ and $d_t = 2d_s$. If $\mathcal{S}$ is $1/n$ permutation fair, then $\mathbf{t}$ is the string representation of a set of d_t -sided dice that is $2/n$ permutation fair.*

Proof. Consider a pattern $\mathbf{x}$ that has $|\mathbf{x}| = 2$ and let $\mathbf{x}'$ be the reverse of $\mathbf{x}$. We have

$$\begin{aligned} (\mathbf{x}; \mathbf{t}) &= (\mathbf{x}; \mathbf{s}) + (\mathbf{x}_{\leq 1}; \mathbf{s})(\mathbf{x}_{> 1}; \mathbf{s}') + (\mathbf{x}; \mathbf{s}') \\ &= (\mathbf{x}; \mathbf{s}) + d_s^2 + (d_s^2 - (\mathbf{x}; \mathbf{s})). \end{aligned} \quad (8)$$

Let $d_t = 2d_s$, and notice that if $\mathcal{T}$ is the set of dice with string representation $\mathbf{t}$, then $\mathcal{T}$ is comprised of d_t -sided dice. Therefore, we have $(\mathbf{x}; \mathbf{t}) = d_t^2/2$ and the result follows from the definition of $2/n$ permutation fairness. ■

Along similar lines, our next theorem provides a way to construct a set of $4/n$ permutation-fair dice out of two sets of $3/n$ permutation-fair dice. As before, this construction works by concatenating a string with its reverse to form a palindrome. Notice that this result demonstrates the advantage of our construction over the concatenation construction as described in [14]. That construction would require the use of $n!$ sets of $3/n$ permutation-fair dice in order to create a set of $4/n$ permutation-fair dice while we are able to use just two sets, independent of the value of n .

Theorem 6. Suppose that $\mathcal{S}$ is a set of d_s -sided dice that has string representation $\mathbf{s} = (s_1, s_2, \dots, s_r)$. Let $\mathbf{t} = \mathbf{s} \parallel \mathbf{s}'$ and $d_t = 2d_s$. If $\mathcal{S}$ is $3/n$ permutation fair, then $\mathbf{t}$ is the string representation of a set of d_t -sided dice that is $4/n$ permutation fair.

Proof. It suffices to show that $(\mathbf{x}; \mathbf{t}) = d_t^4/24 = 8d_s^4/12$ for all $\mathbf{x}$ with $|\mathbf{x}| = 4$. As in the proof of Theorem 2, we can decompose $(\mathbf{x}; \mathbf{t})$ grouping the appearances of $\mathbf{x}$ according to which part of $\mathbf{t}$ each part of $\mathbf{x}$ appears in. Doing so, we see that

$$\begin{aligned} (\mathbf{x}; \mathbf{t}) &= \sum_{i=0}^4 (\mathbf{x}_{\leq i}; \mathbf{s})(\mathbf{x}_{> i}; \mathbf{s}') \\ &= (\mathbf{x}; \mathbf{s}) + (\mathbf{x}; \mathbf{s}') + \sum_{i=1}^3 (\mathbf{x}_{\leq i}; \mathbf{s})(\mathbf{x}_{> i}; \mathbf{s}') \\ &= (\mathbf{x}; \mathbf{s}) + (\mathbf{x}; \mathbf{s}') + \sum_{i=1}^3 \frac{d_s^i}{i!} \frac{d_s^{4-i}}{(4-i)!} \\ &= (\mathbf{x}; \mathbf{s}) + (\mathbf{x}; \mathbf{s}') + \frac{7d_s^4}{12}. \end{aligned} \tag{9}$$

So, it suffices to show that $(\mathbf{x}; \mathbf{s}) + (\mathbf{x}; \mathbf{s}') = d_s^4/12$. Without loss of generality, we can assume that $\mathbf{x} = \mathbf{abcd}$. Notice that we have

$$d_s^4 \mathbf{P}\{\mathbf{a} < \mathbf{b} < \mathbf{c}\} = (\mathbf{dabc}; \mathbf{s}) + (\mathbf{adbc}; \mathbf{s}) + (\mathbf{abdc}; \mathbf{s}) + (\mathbf{abcd}; \mathbf{s}) \tag{10}$$

$$\begin{aligned} d_s^4 \mathbf{P}\{\mathbf{a} < \mathbf{b}, \mathbf{d} < \mathbf{c}\} &= (\mathbf{dabc}; \mathbf{s}) + (\mathbf{adbc}; \mathbf{s}) + (\mathbf{abdc}; \mathbf{s}) \\ &\quad (\mathbf{adcb}; \mathbf{s}) + (\mathbf{dacb}; \mathbf{s}) + (\mathbf{dcab}; \mathbf{s}) \end{aligned} \tag{11}$$

$$d_s^4 \mathbf{P}\{\mathbf{a} < \mathbf{b} < \mathbf{c}\} = (\mathbf{adcb}; \mathbf{s}) + (\mathbf{dacb}; \mathbf{s}) + (\mathbf{dcab}; \mathbf{s}) + (\mathbf{dcba}; \mathbf{s}). \tag{12}$$

Observe that $(\mathbf{x}; \mathbf{s}') = (\mathbf{x}'; \mathbf{s})$. So, we have

$$\begin{aligned} (\mathbf{x}; \mathbf{s}) + (\mathbf{x}; \mathbf{s}') &= (\mathbf{abcd}; \mathbf{s}) + (\mathbf{dcba}; \mathbf{s}) \\ &= d_s^4 (\mathbf{P}\{\mathbf{a} < \mathbf{b} < \mathbf{c}\} + \mathbf{P}\{\mathbf{a} < \mathbf{b} < \mathbf{c}\} - \mathbf{P}\{\mathbf{a} < \mathbf{b}, \mathbf{d} < \mathbf{c}\}) \end{aligned} \tag{13}$$

Finally, because $\mathbf{s}$ is $3/n$ permutation fair we have $\mathbf{P}\{\mathbf{a} < \mathbf{b} < \mathbf{c}\} = \mathbf{P}\{\mathbf{d} < \mathbf{c} < \mathbf{b}\} = 1/6$ and $\mathbf{P}\{\mathbf{a} < \mathbf{b}, \mathbf{d} < \mathbf{c}\} = \mathbf{P}\{\mathbf{a} < \mathbf{b}\} \mathbf{P}\{\mathbf{d} < \mathbf{c}\} = 1/4$. Therefore, we can conclude that $(\mathbf{x}; \mathbf{s}) + (\mathbf{x}; \mathbf{s}') = d_s^4/12$ as required. ■

Numerical Techniques In cases where we have not been able to discover any applicable analytic techniques, we need an efficient way to search for compatible sets of dice numerically. Fortunately, we can characterize compatible sets of dice as the solution to an integer linear programming problem. This allows us to take advantage of a mature ecosystem of software tools designed to solve such problems.

Recall that an integer linear programming problem is a linear optimization problem in which the variables are constrained to be integers. That is, given a set of variables $\mathbf{y} = \{y_i\}_{i=1}^k$, we want to find values of $\mathbf{y}$ that optimize some linear objective function $\mathbf{c} \cdot \mathbf{y}$ subject to the constraints $\mathbf{A} \cdot \mathbf{y} + \mathbf{r} = \mathbf{b}$, $\mathbf{r} \geq 0$, and $\mathbf{y} \in \mathbb{N}^k$. Throughout, we will assume that our ILP solver is configured to minimize the objective function.

We will assume that we have identified a collection $\{\mathbf{s}_i\}_{i=1}^k$ of k candidate sets of dice, all of which are m/n permutation fair. Frequently, the collections that we consider will be comprised of all $n!$ possible permutations of the labels assigned to the dice in a single seed set of m/n permutation fair dice, but we can apply this technique with arbitrary collections of suitable candidates. The variables $\{y_j\}_{j=1}^k$ in our ILP will represent the number of copies of each candidate that we will combine via concatenation to form a new set of dice.

For our objective function, we will let $\mathbf{c}$ be the vector of all ones. That is, $\mathbf{c} = \mathbf{1}$ and the objective function simply counts how many candidates are used in any solution. As such, any solutions that we find will be those that use the fewest possible candidates and the dice in the resulting sets will be as small as possible.

For our constraints, we will let $\mathbf{r}$ and $\mathbf{b}$ be vectors of all zeros. That is, $\mathbf{r} = \mathbf{b} = \mathbf{0}$. All of the permutation fairness constraints will be encoded in the matrix $\mathbf{A}$. Because we are hoping to create a set of dice that are $(m+1)/n$ permutation fair, we will need to consider the counts of all patterns $\mathbf{x}$ with $|\mathbf{x}| = m+1$. If we let r denote the number of such patterns, then $r = \binom{n}{m+1} \cdot (m+1)!$.

Finally, we will let set each column of $\mathbf{A}$ to be the vector of pattern counts for one of the candidate sets of dice that has been normalized to have mean zero. If we let μ_j be the mean value for the j th column of $\mathbf{A}$ then we have $\mu_j = d_j^{m+1}/r$. So, $\mathbf{A}$ will be an $r \times k$ matrix and i, j we will set $a_{ij} = (\mathbf{x}_i; \mathbf{s}_j) - \mu_j$.

All together, this yields a integer linear programming problem in which we want to minimize the quantity $\sum_j y_j$ subject to the constraints $\mathbf{A} \cdot \mathbf{y} = \mathbf{0}$ where $\mathbf{y} \in \mathbb{N}^k$. Incidentally, this formulation shows that the search for compatible sets of dice can be understood as a kind of multi-dimensional knapsack problem.

6. BUILDING SETS OF PERMUTATION-FAIR DICE We can use the machinery developed in the preceding discussion to construct relatively small sets of permutation-fair dice. Starting with some seed set of dice, we can recursively apply Theorems 2, 5, or 6 to build a series of m/n permutation-fair sets of dice, where m increases at each step. This process culminates with a set of dice that are n/n permutation fair.

Example. Consider the set $\mathcal{G}$ of permutation-fair dice described in Table 1. The following four-step recipe describes how our construction can be used to create $\mathcal{G}$.

1. Start with the seed set $\mathcal{S}$ of dice with string representation $\mathbf{s} = \text{abcd}$. Notice that $\mathbf{s}$ is a set of four 1-sided dice that is $1/4$ permutation fair.
2. Let $\mathcal{T}$ be the set of dice with string representation $\mathbf{t} = \mathbf{s} \parallel \mathbf{s}' = \text{abcd} \text{dcb a}$. Theorem 5 implies that $\mathcal{T}$ is a set of four 2-sided dice that is $2/4$ permutation fair.
3. Let $\mathcal{U}$ be the set of dice with string representation $\mathbf{u} = \sigma_1(\mathbf{t}) \parallel \sigma_2(\mathbf{t}) \parallel \sigma_3(\mathbf{t})$, where $\sigma_1 = (\mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{d})$, $\sigma_2 = (\mathbf{d}, \mathbf{b}, \mathbf{a}, \mathbf{c})$, and $\sigma_3 = (\mathbf{c}, \mathbf{b}, \mathbf{a}, \mathbf{d})$. Theorem 2 implies that $\mathcal{U}$ is a set of four 6-sided dice that is $3/4$ permutation fair.
4. Let $\mathcal{G}$ be the set of dice with string representation $\mathbf{g} = \mathbf{v} \parallel \mathbf{v}'$. Theorem 6 implies that $\mathcal{G}$ is a set of four 12-sided dice that is $4/4$ permutation fair.

To create a set of $5/5$ permutation-fair dice, we can follow a recipe like the one described above for creating a set of $4/4$ permutation fair dice. The process is similar, but when we use Theorem 2 to lift $\mathcal{T}_5$ and create a set of $3/5$ permutation fair dice $\mathcal{U}_5$ we find that we need to use six different permutations to do so. As such, at the end of the fourth step, we end up with a set $\mathcal{G}_5$ of five 24-sided dice. Unfortunately, we could not find a way to lift $\mathcal{G}_5$ using less than 120 permutations. Doing so yields a set of five 2880-sided dice that is $5/5$ permutation fair.

Fortunately, we can do better. Rather than starting with a seed that is $1/5$ permutation fair and building everything from scratch, we can start with any seed that is $4/5$ permutation fair and use it to try find a compatible set of dice.

Example. Consider the set $\mathcal{H}$ of five 12-sided dice with string representation

$\mathbf{h} = \text{eeabccddcbadbaccabdcaddabceeeeeeeecbaddabdcdbaccabdabccddcbacee}.$

We can verify by direct computation that $\mathcal{H}$ is $4/5$ permutation fair. So, if we can identify a collection of permutations σ_i such that $\{\sigma_i(\mathbf{h})\}$ are compatible, then we can use Theorem 2 to lift $\mathbf{h}$ and create a set of dice that is $5/5$ permutation fair. Using an ILP solver, we were able to identify the following collection of ten permutations.

$$\begin{array}{ll} \sigma_1 = (a, c, b, d, e) & \sigma_6 = (b, c, a, e, d) \\ \sigma_2 = (d, c, b, a, e) & \sigma_7 = (e, c, a, d, b) \\ \sigma_3 = (e, c, b, d, a) & \sigma_8 = (d, c, a, e, b) \\ \sigma_4 = (d, c, b, e, a) & \sigma_9 = (a, e, d, b, c) \\ \sigma_5 = (e, c, a, b, d) & \sigma_{10} = (b, e, d, a, c) \end{array}$$

So, as we can verify via direct computation, the set $\mathcal{I}$ of dice with string representation

$\text{eeacbddbcabcbdaadcbdbcabbacdeeeeeeeedcabbacdbcbdaadcbacbddbcaee}$
 $\text{eedcbaabcbcbcadddacbacdbbbcaeeeeeeeeacdbbdcabcaddacbdcbabcbdee}$
 $\text{aaecbddbcbcbcedecbdcbecbbceddaaaaaaadcebbecdbcbcedecbdcbcbceaa}$
 $\text{aadcbecbcbcbcedecbdcbecbbceddaaaaaaaecdbbdcbbcedecbdcbcbceebcbdaa}$
 $\text{ddcbabaceacbeebcabceaaecbdddccccdbceaaecbacbeebcaecabbacedd}$
 $\text{ddcbaceacbeebcabceaaecbdddccccdbceaaecbacbeebcaecabbacedd}$
 $\text{bbecaddaceacdeedcadceaaecdbbbbbbdbceaaecdacdeedcaecaddacebb}$
 $\text{bbdcaeeacdaceddecaecdaadcebbbbbdbceaaecdacdeedcaecaddacebb}$
 $\text{ccaedbbdeadebaabedbeaddaebccccccccbeaddaebdebaabedaedbbdeacc}$
 $\text{ccbedaadebdeabbaedaebddbeaccccccccaebddbeadeabbaedbedaaebccc},$

is a set of five 120-sided dice that is $5/5$ permutation fair.

7. FUTURE WORK There are many questions about permutation-fair dice that remain unanswered. Most obviously, perhaps, is the question of whether there exists a permutation-fair set of five d -sided dice where $d \in \{30, 60, 90\}$. Outside of a few highly structured subclasses of dice for which the existence of such sets has been ruled out, this remains a tantalizing open problem.

We can ask similar questions about the existence of permutation-fair dice for other values of n . In particular, one might hope to find a set of six 30-sided dice that is permutation fair. For larger values of n the number of sides required for a set to be permutation fair grows quite quickly, but there is plenty of scope for research into what sorts of bounds might be achievable.

To answer these questions, it would be useful to know which sets of dice can be lifted efficiently. That is, for which sets of dice can we apply Theorem 2 using only a small number of permutations? Can we improve on the asymptotic performance of the best known construction methods for arbitrarily large sets of permutation-fair dice?

If we expand our search to include inhomogeneous sets of dice, then there are even more questions that remain unanswered. In this regime, we have several ways to measure the size of a set of dice. Because the dice in such sets are not all the same size, we might consider only the size of the largest die. Alternatively, we might consider the total number of faces on all dice in the set. How small can a set of permutation-fair dice be when we use one of these metrics to measure its size?

ACKNOWLEDGMENT. While the work described herein was done independently, after finding the permutation fair set of five 120-sided dice, we shared our results with Eric Harshbarger. This has since led to a wonderful series of collaborations both with him and other researchers. We would like to thank him for his pioneering technical contributions and for the work that he has done to organize the community of researchers interested in permutation-fair dice. We would also like to thank all of the reviewers, particularly Michael Cromer and Brett Witty, for their excellent feedback throughout the drafting of this paper.

REFERENCES

1. Buhler J., Graham R., Hales A. (2018). Maximally nontransitive dice. *Amer. Math. Monthly*. 125(5): 387-399. doi.org/10.1080/00029890.2018.1427392
2. Forrest, J., Ralphs T., Santos, H.G., Vigerske, S., Hafer, L., Kristjansson, B., Lubin, M., Brito, S., Saltzman, M., Pitrus, B., et al. (2022). *coin-or/Cbc*, Version 2.10.8. Zenodo. doi.org/10.5281/zenodo.6522795
3. Cooley, C., Ella, W., Follett, M., Gilson, E., Traldi, L. (2014). Tied dice. II. Some asymptotic results. *J. Combin. Math. Combin. Comput.* 90: 241-248.
4. Diaconis, P., Keller J.B. (1989). Fair Dice. *Amer. Math. Monthly*. 96(4): 337-339. doi.org/10.2307/2324089
5. Dunning, I., Mitchell, S., O'Sullivan, M. (2011). PuLP: A linear programming toolkit for Python.
6. Ford, R., Grime, J., Harshbarger, E., Pollock, B. (2023). Go first dice for five players and beyond. *Recreat. Math. Mag.* 10: 75-87. doi.org/10.2478/rmm-2023-0004
7. Gardner, M. (1970). The paradox of the nontransitive dice and the elusive principle of indifference. *Sci. Amer.*, 223(6): 110-114. <https://www.scientificamerican.com/article/mathematical-games-1970-12/>
8. Grime, J. (2017). The bizarre world of nontransitive dice: games for two or more players. *College Math. J.* 48(1): 2-9. doi.org/10.4169/college.math.j.48.1.2
9. Grünbaum, B., Shephard, G.C. (1981). Spherical tilings with transitivity properties. In: Davis, C., Grünbaum, B., Sherk, F.A. (eds) *The Geometric Vein*. New York, NY: Springer, pp. 65-98.
10. Harshbarger, E. (2018). A method to construct $n + 1$ -player permutation-fair sets of dice from known n -player permutation fair sets.
11. Harshbarger, E. (2023). Go first dice. Website. http://www.ericharshbarger.org/dice/go_first_dice.html.
12. Hur, I., Kim, Y. (2021). Possible probability and irreducibility of balanced nontransitive dice. *J. Math.* Vol. 2021, Article ID 6648248. doi.org/10.1155/2021/6648248
13. Kronenthal, B.G., Traldi, L. (2009). Tied dice. *J. Combin. Math. Combin. Comput.* 68: 85-96.
14. Polylog. (2022) We designed special dice using math, but there's a catch. YouTube. <https://www.youtube.com/watch?v=-64UT8yikng&t=66s>
15. Polymath 13 (2017). The probability that a random triple of dice is transitive. Website. <https://polymathprojects.org>.
16. Schaefer, A. (2017). Balanced non-transitive dice II: tournaments. arXiv:1706.08986 [math.CO], Jun. 2017. doi.org/10.48550/arXiv.1706.08986
17. Schaefer, A., Schweig, J. (2017). Balanced nontransitive dice. *College Math. J.*, 48(1):10-16. doi.org/10.4169/college.math.j.48.1.10
18. Steinhaus, H., Trybula, S. (1959). On a paradox in applied probabilities. *Bull. Pol. Acad. Sci. Math.*, 7: 67-69.
19. Trybula, S. (1961). On the paradox of three random variables. *Zastos. Mat.*, 5: 321-332.
20. Trybula, S. (1965). On the paradox of n random variables. *Zastos. M.*, 8: 143-156.

MICHAEL PURCELL received his PhD from the University of Utah in 2010 where he studied under the supervision of Davar Khoshnevisan. After completing his degree, he worked as an applied research mathematician for the U.S. Department of Defense for ten years. In 2020 he joined the Software Innovation Institute at The Australian National University as a senior research engineer. In 2022 he moved into his current role as a Senior Fellow in the School of Computing at the Australian National University. His research interests include probability, cryptography, and differential privacy. In addition to his technical work, he is also an enthusiastic amateur musician and game designer.

School of Computing, The Australian National University, Canberra ACT 2601, Australia
michael.purcell1@anu.edu.au